ÖZET RAPOR · 28 Eylül 2026

Gerçek Zamanlı Stres Tespitinde Açıklanabilirliğin Hesaplama Maliyeti

Yöntem Envanteri ve Ölçüm Sonuçları

Bu rapor kimin için. Makine öğrenmesi ve XAI kavramlarını bildiğin varsayılıyor — SHAP, LIME, faithfulness, ECE gibi terimler tanımlanmıyor, yalnızca *hangi varyantı, hangi ayarla* kullandığımız yazılıyor. Buna karşılık **haberleşme ve dağıtık sistem tarafı** (Bölüm 3) ayrıntılı anlatılıyor, çünkü işin yeni olduğun kısmı orası.

Yapı. Bölüm 2 projede kullanılan *bütün* yöntemlerin envanteri; Bölüm 3 haberleşme altyapısı ve yedi metrik; Bölüm 4 sonuçlar; Bölüm 5 uyarılar. Her yöntem yalnızca bir yerde anlatılır, sonuç bölümünde tekrar edilmez.

Ayrıntılı sürüm. Her deneyin tam dökümü, tam sözlük, geçersiz sayılan ölçümlerin gerekçeleri ve bitirme projesi sonuçları: `docs/teknik_rapor.pdf` (53 sayfa). Bütün sayılar aynı kaynaklardan gelir.

1 Ana bulgular

Tek cümlelik sonuç

Gerçek zamanlı bir stres tespit sisteminde darboğaz **tahmin değil, açıklamadır**. Sis düğümünde bir 10 saniyelik pencerenin tahmini 3,39 ms, aynı pencerenin SHAP/LIME açıklaması 395–1422 ms sürüyor — toplam hesabı **118–421 kat** büyütüyor. Bunun sonucu: sistem tasarımındaki kritik karar *"hesabı nerede yapayım"* (uç/sis/bulut) olmaktan çıkıp *"ne zaman açıklayayım"* olmaya kayıyor.

Bulgu	Sayı	Kaynak
Açıklamanın toplam hesabı büyütme oranı	118–421×	`04_fog/`
Yerleşim farkı, açıklama *kapalıyken* (bulut/uç)	8,7×	`13_bulut/`
Yerleşim farkı, açıklama *açıkken* (bulut/uç, SHAP)	**1,13**×	`13_bulut/`
Seçici açıklamanın ortalama yanıtta kazancı	5,6×	`09_secici_xai/`
Seçici açıklamanın kapasitede kazancı	7,5×	`10_yuk/`
Projeksiyon ile ölçümün uyumu	%2	`09_secici_xai/`

Önerilen çözüm ve ölçülen kazancı

Güven eşikli seçici açıklama: model kendinden eminse (güven $\geq 0{,}80$) sadece tahmin döndür; emin değilse tahmin + açıklama döndür. **1030 pencerelik** gerçek bir koşuda ölçüldü: ortalama yanıt süresi $450{,}5 \rightarrow 80{,}1$ ms (**5,6**×), pencerelerin yalnızca **%14,1**'i açıklanıyor, buna rağmen yanlış sınıflanan pencerelerin **%89,7**'si açıklama alıyor, tek işçili kapasite $12 \rightarrow 90$ eşzamanlı istemci.

2 Kullanılan yöntemler envanteri

Bu bölüm "projede ne yaptık" sorusunun tam cevabıdır. Her satır: yöntem, nerede kullanıldığı, ve varsa ölçülen değeri.

2.1 Veri ve sinyal işleme

Yöntem	Nerede	Ne yapıldı / ölçüldü
Çok kanallı birleştirme	`data/loader.py`	Göğüs 8 + bilek 6 = 14 kanal. **Sabit sıra zorunlu**: sıra bozulunca doğruluk 1,00'den 0,52'ye düştü (gözlendi).
Ham hızda (native) akış	`NATIVE_STREAMS`	Göğüs 700 Hz; bilek ACC 32 Hz, BVP 64 Hz, EDA/TEMP 4 Hz. Her akış kendi hızında taşınır.
Doğrusal interpolasyonla yeniden örnekleme	`np.interp`	Hepsi ortak 64 Hz'e indirilir. **Pencere başına, sunucuda** yapılır, yani maliyeti ölçülür: **1,57 ms = toplamın %28,6'sı**.
Kayan pencere	`MultiRateBuffer`	10 s = 640 örnek, hop 5 s = 320 örnek, **%50 örtüşme** → her 5 saniyede bir yeni tahmin.
İki tampon stratejisi	`streaming/buffer.py`	`RollingBuffer` (indirgeme ölçüm dışında) ile `MultiRateBuffer` (indirgeme ölçüm içinde) karşılaştırıldı.
Tick tabanlı replay	`streaming/`	500 ms'lik bloklar hâlinde gerçek zamanlı sensör akışı taklidi.
Çoğunluk etiketi	pencereleme	Geçersiz etiketli pencereler **işlenir** ama rapordan çıkarılır. Eskiden atlanıyordu; bu normalizasyon istatistiğini bozuyordu, düzeltilince doğruluk 0,7674 → **0,8222**.
İkili etiketleme	`config`	Stres (2) ile stres-değil (1, 3, 4: taban çizgi, eğlence, meditasyon).
Alt-örnekleme	`data/windowing.py`	Yalnızca offline eğitimde, sınıf dengesi için.

2.2 Model ve eğitim

Yöntem	Nerede	Detay
1B Temporal CNN	`models/temporal_cnn.py`	Conv1d → BatchNorm → ReLU → MaxPool(2) → Dropout, iki blok; sonra GAP → FC(128→64) → BN → ReLU → Dropout → FC(64→2).
Mimari seçimi	depth 2, [32,128], k9	**49 890 parametre**, 14,50M MAC/pencere, tepe aktivasyon 81 920 eleman (`bn2`).
Focal loss	eğitim	$\alpha = 0{,}5$, $\gamma = 2$ — sınıf dengesizliği için.
Adam + weight decay	eğitim	lr 1e-3, wd 1e-4, batch 64.
Erken durdurma	eğitim	150 epoch üst sınır, patience 20, seed 42.
LOSO protokolü	değerlendirme	15 katlı. Kanonik sayı **F1 0,92–0,93**. (Checkpoint'teki 0,9386 yalnızca iki denek üzerinde ve LOSO değil — makalede kullanılmamalı.)
Karşılaştırma modeli	`Transformer.ipynb`	TranSenseFusers: Temporal CNN + 8 başlı çapraz dikkat (dim 64), **1 859 026 param, 103,7 ms çıkarım** → CNN'e göre 37,3× parametre, 28,6× çıkarım.

Yöntem	Nerede	Detay
Checkpoint'ten mimari okuma	`load_checkpoint()`	Mimari config'ten değil checkpoint'in kendi meta alanından okunur — sessiz mimari uyuşmazlığı imkânsız (`strict=True`).

Model seçiminin gerekçesi ve iki dürüstlük notu

Bitirmede en yüksek doğruluğu TranSenseFusers verdi (%91,13) ama dağıtım için 49,9K parametreli CNN dondurulduğu — gerekçe gecikme.
1. %91,13 *LOSO değil*: tek test deneği (S3), doğrulama S16 (F1 1,0000). Makalede bu sayıya "LOSO" denmemeli. **2.** Pencere hızı da farklı: transformer 700 Hz'de 7000 örnek, CNN 64 Hz'de 640 örnek — 28,6× farkın bir kısmı buradan geliyor, tabloda "sampling rate" sütunuyla gösterilmeli.

2.3 Normalizasyon

Yöntem	Kullanım	Ölçülen
Denek bazlı z-score	offline eğitim	Referans. Gerçek zamanlıda **kullanılamaz** — geleceğe sızma.
Nedensel z-score (Welford)	`running`, **varsayılan**	Sadece o ana kadarki veriden çevrimiçi ortalama/std. Güncelleme maliyeti dürüstlük için `preprocessing`'e eklenir (**0,25 ms**).
Sabit z-score	`fixed`	Yalnızca eğitim deneklerinden tek bir mean/std; en katı dağıtım senaryosu.
Sızıntı miktarının ölçümü	doğrulama	İki girdi uzayı arasında bağıl L2 farkı **0,787 (S11) / 0,773 (S2)**, standart sapma 1,00'e karşı 1,25.

Ölçülen bedel: offline weighted F1 0,9386 → streaming doğruluk 0,8222. Düşüş modelin değil, *normalizasyon kısıtının* sonucu — makalede bu cümle olmalı.

2.4 Açıklanabilirlik yöntemleri

Yöntem	Nerede	Detay
SHAP **GradientExplainer**	`xai/shap_.py`	Gradyan tabanlı, modelin içine erişir. **KernelSHAP değil** — literatürdeki "SHAP yavaştır" iddiası ona aittir, bizim sayılarımıza değil.
k-means ile arka plan seçimi	`xai/artifacts.py`	2355 pencerelik havuzdan **100 pencere**, S2–S5'ten (test denekleriyle kesişmiyor), seed 42, nedensel uzayda.
Arka planın dondurulması	`.npz` + `xai_config.json`	Diskten okunur ve **SHA-256 doğrulanır**. Sunucu her açılışta yeniden kümelemez; tarama ile bit-birebir aynı sonuç garanti.
`nsamples` çalışma noktası	SHAP	200 (tam) ve **50** (ucuz).

Yöntem	Nerede	Detay
Bölge tabanlı LIME	`xai/lime_.py`	Hazır `lime` paketi **kullanılmadı**. Görüntüdeki süperpikselin karşılığı: 20 zaman dilimi × 14 kanal = **280 bölge**.
Rastgele ikili maske örnekleme	LIME	250 / 500 perturbasyon, seed sabit; ilk örnek hep orijinal pencere.
Kanal ortalamasıyla maskeleme	LIME + kalite metrikleri	Sıfırlamak yerine. Normalize sinyalde sıfır "bilgi yok" değil "kaydın ortalaması" demektir ve fizyolojik olarak anlamsızdır.
Üstel kernel ağırlıklandırma	LIME	$w = \sqrt{\exp(-d^2/kw^2)}$, $kw = 0{,}5$; d = kapalı bölge oranı.
Ağırlıklı Ridge regresyon	LIME	$\alpha = 1{,}0$; katsayılar doğrudan atıf haritası olur.
Partili ileri geçiş	`xai/base.py`	chunk 256. `model.eval()` olduğu için parti boyutu sonucu değiştirmez — BatchNorm koşu istatistiklerini kullanır.
Bölge ızgarası önbellekleme	LIME	Her çağrıda yeniden kurmak, ölçülen gecikmeye modelin işi olmayan bir maliyet eklerdi.
Açıklayıcı ısıtma (warm-up)	`serving/server.py`	SHAP **ilk** çağrı 12,4 s, sonraki 0,85 s. Isıtılmasa tek worker'ı 12 saniye bloklar, bütün kuyruk arkasında birikirdi. Ölçülen ısınma: 1208–1251 ms.
Seed sabitleme	her iki yöntem	Sabitlenmeden önce SHAP sensitivity 0,139'du, sonra **0,075** — farkın neredeyse yarısı yöntemin kendi gürültüsüymüş.
OFAT taraması	Colab / GPU, `wesad_xai_ofat_sweep_colab.py`	Her aşamada tek parametre değiştirilir: LIME 9 yapılandırma (örnek sayısı, bölge sayısı, kernel genişliği), SHAP 8 (arka plan boyutu, `nsamples`, seçim yöntemi). Colab/GPU, 40 pencere, nedensel uzay.

Çalışma noktası	Yapılandırma	Sis düğümünde
`shap_cheap`	GradientExplainer, arka plan 100 (kmeans), nsamples 50	395 ms
`lime_cheap`	bölge tabanlı, 280 bölge, 250 örnek, kw 0,5	590 ms
`lime`	aynı, 500 örnek	1189 ms
`shap`	aynı, nsamples 200	1422 ms

Tablo 5: Taramadan **elle** seçilen dört çalışma noktası. Otomatik seçim yalnızca faithfulness'a baktığı için kullanılmadı — gerekçesi `xai_config.json`'a yazıldı.

2.5 Açıklama kalitesi ölçütleri

Ölçüt	Nasıl hesaplanıyor	Ölçülen
Deletion eğrisi + AUC	En önemliden başlayarak kapat, olasılığı izle; 20 kontrol noktası, trapez integral	Düşük AUC iyi
Insertion eğrisi + AUC	Taban çizgiden başlayıp en önemliyi ekle	Yüksek AUC iyi
Faithfulness	insertion AUC − deletion AUC	**0,209–0,263** — dört nokta arasında **fark yok** (SEM ≈0,04)
Monotonicity	Insertion'da azalmayan adım oranı (yüksek iyi)	0,549–0,604
Sensitivity	Gürültüye karşı normalize L2 değişim ($0{,}05 \times \sigma$, 3 deneme, düşük iyi)	LIME **0,045–0,049** vs SHAP **0,123–0,128** → LIME 2,7× kararlı, fark hata payının 15 katı
Self-instability	Aynı pencere iki kez açıklanınca fark	Seed sabitken ≈0, her iki yöntemde
Kanal ortalaması taban çizgi	Ölçüt ile yöntem tutarlı olsun diye açıklayıcıların kendi perturbasyonuyla aynı	—
SEM	$\sigma/\sqrt{n}$, $n = 40$ pencere	Taramanın ilk hâlinde **yoktu**; kazananlar 0,003'lük farkla seçiliyordu. Eklenince o seçimin geçersiz olduğu anlaşıldı.
Pareto analizi	`tools/pareto.py`	Kalite bu makinede (donanımdan bağımsız), gecikme sis düğümünden (dağıtım donanımı)

2.6 Güven kalibrasyonu

Yöntem	Nasıl	Sonuç
Reliability diyagramı	15 eşit genişlikli kutu	Şekil 4
ECE / MCE	Kutu ağırlıklı ve en büyük |güven − doğruluk|	S11 **0,047** / 0,252 · S14 **0,227** / 0,522
Brier / NLL	Olasılık tahmininin kare hatası / log kaybı	S11 0,038 / 0,138 · S14 0,266 / 0,898
Fazla özgüven	ortalama güven − doğruluk	S11 −0,043 · S14 +**0,218**
Sıcaklık ölçekleme	NLL minimizasyonu ile tek parametre T	S11'de T=3,48, S14'te T=0,58
Çapraz denek uygulama	Bir denekte öğren, diğerinde uygula	**Çalışmadı**: her iki yönde de kötüleştirdi
Güven yüzdelikleri	p1, p5, p25, p50, p75, p95, p99	S11 p25 = 0,918 → düşük güven nadir; seçici açıklamanın kazancı buradan gelir

2.7 Seçici açıklama (ana katkı)

Yöntem	Detay
Güven eşikli tetikleme	`should_explain(conf, τ)`: `conf` $< \tau$ ise açıkla. τ=0 hiç (XAI'siz taban), τ=1 her pencere (en pahalı durum), arası seçici. 1,0 ayrıca ele alınır çünkü softmax tam 1,0 okuyabiliyor.
Aynı worker içinde üretim	Açıklama, tahminle aynı worker'da ve `execution time`'ın *içinde* çalışır. Ayrı ipliğe atılsaydı maliyeti görünmez olurdu — deneyin bütün amacı o maliyeti ölçmek.
Eşik taraması	4 aday: 0,60 / 0,70 / 0,80 / 0,90, iki denekte.
Üç karar ölçütü	`explain_rate` (maliyet), `error_recall` (fayda), `precision_of_trigger` (israf: açıklananların kaçı gerçekten hatalıydı).
Karışım (mixture) modeli	ort $= (1-p) \cdot$ taban $+ p \cdot$ açıklamalı. 4 açıklayıcı $\times$ 2 denek $\times$ 6 eşik = 48 satırlık projeksiyon (`tools/selective_sweep.py`).
Projeksiyonun ölçümle sınanması	Ölçülen tek nokta (τ=0,80, S11) ile tahmin karşılaştırıldı $\rightarrow$ **%2** uyum.

2.8 Deney tasarımı ve ölçüm metodolojisi

Kural / yöntem	Gerekçe ve ölçülen
Isınma penceresi hariç	İlk **15** pencere hiçbir istatistiğe girmez (önbellek/CUDA ısınması).
Dönüşümlü tekrar	A, B, A, B sırası. Ardışık koşulsaydı işlemci frekans ölçeklemesi son ölçülen yapılandırmayı sistematik olarak yavaş gösterirdi.
İki dağılımın ayrılması	*Koşu içi* (bir koşudaki yüzlerce pencerenin p50/p95/p99 — sistemin davranışı) ile *koşu arası yayılım* (aynı deney tekrarlanınca medyanın oynaması — ölçümün güven aralığı) karıştırılmaz.
p50 / p95 / p99 birlikte	"En hızlı çalışma" hiçbir yerde raporlanmaz.
`range_pct` ve CV	Yayılımın iki ölçüsü: (max$-$min)/medyan ve σ/μ.
Tek iş parçacığı	1/2/4 iplik, 4 tur dönüşümlü taranıp karar verildi. Fark yayılımın altında kaldı; **gerekçe hız değil kararlılık** (yayılım %2,9'a karşı %11–15).
İki ardışık düzen yolu	`prepared` (indirgeme ölçüm dışında) ile `native` (içinde) karşılaştırıldı; fark %38,6, yayılımın (%9,4) çok üstünde, yani gerçek.
Çapraz doğrulama denetimi	Tek süreçli yol ile istemci–sunucu yolu **birebir aynı** sınıflandırma sonucunu veriyor (acc 0,8222, TN 743 / FP 184 / FN 29 / TP 242) $\rightarrow$ ağ katmanı hesabı bozmuyor.
Batch boyutu 1	Gerçek kullanıcı tek pencere gönderir.
Her sonuç JSON'a	Grafikler JSON'dan üretilir (`python -m tools.figures`); elle sayı taşınmaz.

Kural / yöntem	Gerekçe ve ölçülen
Koşu künyesi	Sunucu donanımı, Python/PyTorch sürümü, XAI yöntemi, eşik ve tam yapılandırma her koşuya kaydedilir. **Yazılım yığını farkı bu sayede yakalandı** (Bölüm 5).

2.9 Donanım fizibilite analizi (ölçüm değil, hesap)

`tools/mcu_feasibility.py` modelin bir mikrodenetleyicide çalışıp çalışmayacağını *donanım olmadan* hesaplar.

Yöntem	Sonuç
Katman katman parametre / aktivasyon / MAC sayımı	49 890 parametre, 14 497 984 MAC/pencere
Tepe aktivasyon analizi	81 920 eleman, `bn2` katmanında — darboğaz orada (128 kanal × 320 uzunluk)
Nicemleme senaryoları	fp32 / fp16 / int8 → Flash 194,9 / 97,4 / 48,7 KB; tepe RAM 320 / 160 / 80 KB
MAC/çevrim tabanlı süre tahmini	Cortex-M4@168 MHz 287,7 ms · M7@480 MHz fp32 37,8 ms · M7 int8 12,1 ms — en yavaşı bile 5000 ms bütçenin %5,8'i

Sonuç: MCU'da darboğaz hız değil **bellek**. int8 nicemleme STM32F407/F746 için zorunlu; H743 ve U575 fp32'yi de kaldırır. *Bu analiz kâğıt üstündedir* — gerçek port ONNX aktarımı, nicemleme, ön işleme portu ve cihaz üstü ölçüm gerektirir.

2.10 Bilinçli olarak kullanılmayanlar

Yöntem	Neden yok
KernelSHAP	GradientExplainer seçildi (gradyan erişimi var, çok daha hızlı). Makalede açıkça yazılmalı, yoksa "SHAP yavaş" literatürüyle çelişik görünür.
Integrated Gradients	Bitirme projesinde vardı, bu çalışmanın kapsamı dışında.
Hazır `lime` paketi	8960 değişkeni ayrı ayrı saymak hem pahalı hem yorumlanamaz.
Broker	Tek worker varken dağıtacak iş yok; referansın kendi sonucunda da broker/worker farkı 43,9'a karşı 43,3 ms.
Çok worker	Tek worker varsayımı referansla aynı ve kuyruk metriğinin ön koşulu.
Model karmaşıklığı ablasyonu	OFAT tablosu var ama varyantlar eğitilmedi — Kasım seti.
Kullanıcı çalışması	Standart XAI sınırlılığı.

3 Haberleşme tarafı: altyapı ve ölçüm yöntemleri

Bu bölüm ölçümlerin *neyi* ölçtüğünü anlatır. Sayılara geçmeden önce okunması gereken kısım burasıdır.

3.1 Üç yerleşim ne demek

Yerleşim (placement) — hesabın fiziksel olarak nerede yapıldığı

Uç / ortak yerleşim (edge, co-located)
Sunucu, istemciyle **aynı makinede**. Konuşma TCP *loopback* (127.0.0.1) üzerinden: gerçek ağ kartı ve kablo devrede değil, paket işletim sisteminin içinden döner. Ağ gecikmesi ≈1 ms. *Dikkat:* bu raporda "edge" **giyilebilir cihaz anlamına gelmiyor**; makalede "co-located" demek daha güvenli.

Sis (fog)
Aynı yerel ağdaki ikinci fiziksel makine (192.168.1.14, Intel 12 çekirdek, 16 GB), Wi-Fi üzerinden. Ağ gecikmesi 4–5 ms. Literatürde "fog" = kullanıcıya yakın, sınırlı kaynaklı sunucu (tipik olarak Raspberry Pi). *Buradaki makine güçlü olduğu için ölçülen şey aslında "dağıtık yerleşim", "kaynak kısıtlı sis düğümü" değil* — Bölüm 5.

Bulut (cloud)
İnternetteki uzak VPS (DigitalOcean, 167.71.43.64, **1 çekirdek**, 2 GB), Docker konteynerinde. Ağ gecikmesi 40–60 ms.

3.2 Mimari ve üç tasarım kararı

```
İSTEMCİ (veri burada kalır --- sunucu WESAD'a ihtiyaç duymaz)
  her 500 ms'de bir "tick" bloğu → TCP paketi, t_send damgası
    ↓ AĞ (loopback / Wi-Fi / internet)
SUNUCU — okuyucu görevi: paketi al, t_recv damgala, KUYRUĞA koy
    ↓ (kuyrukta bekleme = queuing delay)
SUNUCU — TEK WORKER: t_start → [resample → z-score
           → CNN → güven < eşik ise AÇIKLAMA] → t_end
    ↓ cevap paketi (pred, confidence, stages, explained)
İSTEMCİ: t_reply − t_send = yanıt süresi
```

1. **Açık istek kuyruğu + tek worker.** İlk sürümde paket okunur okunmaz aynı yerde işleniyordu. O zaman `t_start` ile `t_recv` bitişik olduğu için *kuyruk gecikmesi her zaman sıfıra yakın okuyordu* ve bekleme süresi ağ gecikmesinin içinde gizleniyordu. Şimdi okuyucu görev paketi alıp varış zamanını damgalıyor ve ortak kuyruğa koyuyor; tek worker kuyruğu sırayla boşaltıyor. Sonuç: iki istemciyle p99 kuyruk gecikmesi $0{,}37 \rightarrow 5{,}50$ ms — metrik artık gerçekten bekleme ölçüyor.
2. **Açıklama, yürütme süresinin içinde** (Bölüm 2.7).
3. **Broker yok** (Bölüm 2.10).

Altyapı yöntemi	Detay
Özel TCP tel protokolü	`serving/protocol.py`: uzunluk önekli çerçeveleme, ikili paketleme, PING/PONG saat senkronizasyonu
`asyncio` eşzamanlılık	Bağlantı başına okuyucu görev + ortak kuyruk + tek worker görevi
`psutil` ile kaynak örnekleme	0,1 s aralıkla süreç CPU %'si ve RSS; özet mean/max/p95
Sunucu donanım künyesi	Platform, CPU, RAM, Python ve PyTorch sürümü her koşuya yazılır (Raspberry Pi'de `/proc/device-tree/model` da okunur)
Docker + docker-compose	Bulut dağıtımı; `.dockerignore` XAI paketini imajdan dışlar (`--xai-method none` varsayılan olduğu için gerekmiyor)
Ortam değişkeniyle yapılandırma	`WESAD_XAI_METHOD`, `WESAD_XAI_THRESHOLD` — konteyner yeniden inşa edilmeden durum değiştirilebilir

3.3 Ölçülen yedi metrik

Bu yedi metrik referans makalenin (Erzurumluoğlu ve ark., ICACT 2025) ölçtüğü metriklerle birebir aynıdır; karşılaştırma bu yüzden mümkün.

Metrik	Ne ölçer	Hangi saat	Kırılganlık
latency (ağ gecikmesi)	Paketin istemciden sunucuya *tek yönde* ulaşma süresi.	İki ayrı saat	**En kırılgan.** Saat farkı düzeltmesi gerekir; hata payı Wi-Fi'da $\pm$1–1,7 ms, internette $\pm$25–51 ms.
queuing delay	Worker meşgulken kuyrukta bekleme ($t_{recv} \rightarrow t_{start}$).	Sunucu saati	Sağlam — tek saat.
execution time	Ardışık düzenin çalışma süresi: resample + z-score + tensör + çıkarım + son işleme + *(varsa)* açıklama ($t_{start} \rightarrow t_{end}$).	Sunucu saati	Sağlam. **Kapasite hesabının girdisi budur.**
response time	İstemcinin gördüğü toplam süre; yaklaşık $2 \times$ latency $+$ queuing $+$ execution.	İstemci saati	Sağlam. Kullanıcının hissettiği sayı budur.
jitter	Ardışık yanıt sürelerinin ortalama mutlak farkı — sistemin "düzensizliği".	İstemci saati	Sağlam. Wi-Fi'da yüksek çıkar; bu ağın kendi özelliğidir.
bandwidth	(gönderilen + alınan bayt) / süre, kbps.	—	`realtime` kapalıyken **anlamsız** (istemci alabildiğine hızlı gönderir).
resource usage	Sunucu sürecinin CPU %'si ve RSS belleği.	Sunucu tarafı	CPU %100'ün biraz üstü okur (tek çekirdek + örnekleme gürültüsü).

Tablo 10: Pratik kural: iki farklı saat kullanan tek metrik latency'dir; ondan şüphe duyduğunda diğer altısına bakabilirsin.

Yüzdelikler: p50 / p95 / p99

p50 (medyan) değerlerin yarısının altında kaldığı nokta — ortalamadan dayanıklıdır. **p95** ve **p99** "en kötü durum" davranışıdır ve gerçek zamanlı sistemde ortalamadan önemlidir: kullanıcı 100 pencerenin 95'inde hızlı, 5'inde 2 saniye bekliyorsa sistem "yavaş" hissettirir.

3.4 Zaman ve saat ölçümü

Projede en çok zaman alan ve en çok hata üreten kısım burasıydı; makalede metodoloji katkısı olarak anlatılmalı.

1. **Sorun 1: saat çözünürlüğü.** İlk sürüm `time.time_ns()` kullanıyordu; Windows'ta çözünürlüğü ölçüldü: **15,6 ms**. Loopback gecikmesi 0,1 ms mertebesinde olduğu için latency medyanı *tam sıfır* okuyordu — metrik tamamen anlamsızdı. **Çözüm:** `perf_counter_ns()`, 100 ns çözünürlük.
2. **Sorun 2: `perf_counter` sürece özeldir.** Referans noktası her süreçte farklı, yani iki makinenin sayaçları doğrudan karşılaştırılamaz. **Çözüm:** bağlantı başında NTP mantığıyla saat farkı ölçülür — 12 tur PING/PONG, *en düşük gidiş-dönüş süreli tur* seçilir (en az gürültülü ölçüm odur). Belirsizlik = RTT/2 ve rapora yazılır.
3. **Sorun 3: kayma (drift).** İki kristal birebir aynı hızda tiklemez. 87 dakikalık bir Wi-Fi koşusunda kayma latency medyanını $-$**8,75 ms**'ye düşürdü — paket gönderilmeden varamayacağı için fiziksel olarak imkânsız, yani ölçüm geçersizdi. **Çözüm:** saat farkı her **60 saniyede** yeniden ölçülür (`--clock-resync-s`) ve her paketin latency'si iki ölçüm arasında *doğrusal ara değerle* düzeltilir. Ölçüm, son cevap alındıktan sonraki sessiz anda 4 turda yapılır, böylece akışı kesmez.

4. **Tutarlılık kontrolü.** Özette toplam kayma ve kayma hızı (ppm) raporlanır; ayrıca latency medyanı $(\text{response} - \text{execution} - \text{queuing})/2$ ile karşılaştırılır ve negatif medyan görülürse uyarı basılır.

Bağlantı	tipik RTT	latency belirsizliği	ölçülen kayma
loopback (uç)	0,26 ms	±0,13 ms	≈0 ppm
Wi-Fi (sis)	2–3 ms	±1,1–1,7 ms	−5 …−10 ppm
internet (bulut)	50–100 ms	±25–51 ms	−104 …+66 ppm

Tablo 11: Ölçüm kalitesi bağlantı türüne göre. Wi-Fi'da tek yönlü gecikmenin hata payı, ölçülen değerin kendisiyle aynı mertebeye çıkabilir; ölçüm kalitesi önemliyse Ethernet tercih edilmelidir.

3.5 Kapasite ve kuyruk modeli

Kapasite hesabı

Sistem her **5 saniyede** bir yeni pencere üretir (hop süresi). Bir worker, bir pencereyi *execution time* kadar sürede işler. Tek worker'ın taşıyabileceği eşzamanlı istemci sayısının üst sınırı:

$$\text{kapasite} = \frac{\text{hop bütçesi } (5000\,ms)}{\text{execution time}}$$

Bu bir *üst* sınırdır: gerçekte kuyruk dalgalandığı için daha erken tıkanır.

Kuyruk neden beklenenden çok daha kötü davranıyor

4 istemcide teorik doluluk yalnızca %32 iken **ölçülen kuyruk p95 1125 ms**. Sebep ortalama doluluk değil, **eşzamanlı patlama**: istemciler birlikte başlatıldığı için *faz hizalı* kalıyor, pencerelerini aynı anda üretiyor ve tek worker'da açıklamalar arka arkaya diziliyor. Basit model tutuyor:

$$(N-1) \times \text{açıklama süresi} = 3 \times 370{,}75 = 1112\,ms \quad — \quad \text{ölçülen p95} = 1125\,ms$$

Açıklama kapalıyken de tutuyor: $3 \times 3{,}69 = 11{,}1$ vs. ölçülen 12,5 ms. *Bu bir üst sınırdır* — gerçek dağıtımda istemciler rastgele fazlarda başlar; makalede bu cümle mutlaka yazılmalı.

4 Sonuçlar

4.1 Hesaplama profili ve sınıflandırma başarımı

Aşama (p50, ms)	tek süreç	tekrarlı native	tekrarlı prepared	sis uçtan uca
resampling	1,378	1,573 (%3,3)	—	1,044
preprocessing	0,218	0,252 (%3,5)	0,580 (%7,9)	0,172
tensor_prep	0,141	0,149 (%2,7)	0,135 (%5,7)	0,114
inference	2,682	3,039 (%8,9)	2,969 (%9,1)	1,948
postprocessing	0,086	0,087 (%3,0)	0,085 (%5,5)	0,046
toplam	4,689	**5,502 (%8,1)**	3,971 (%9,4)	**3,530**

Tablo 12: Aşama profili dört kaynaktan. Parantez içi *koşu arası yayılım*; **belirsizlik gerektiğinde "tekrarlı native" sütunu kullanılmalı** — yayılım taşıyan tek sütun odur. Sis sütunu düşük çünkü o makine daha hızlı. Kaynak: `12_profil/profile_table_b1.json`

Koşu	denek	n	doğruluk	stres F1	duyarlılık	TN/FP/FN/TP
Tek süreç, native	S11+S14	1198	0,8222	0,6944	0,8930	743/184/29/242
Sis, tam kayıt	S11	604	**0,9520**	**0,8807**	0,7868	468/**0**/29/107

Tablo 13: Sınıflandırma başarımı. S11+S14 birlikte okunduğunda model *stres tarafına eğilimli* (184 yanlış alarma karşılık 29 kaçırma); yalnız S11'de hiç yanlış alarm yok. Bu fark deneğe bağlıdır — kalibrasyon bulgusuyla (Bölüm 4.8) aynı kökten gelir.

İş parçacığı	toplam p50	koşu arası yayılım	toplam p95
1	3,397 ms	**%2,9**	4,980 ms
2	3,933 ms	%14,7	5,774 ms
4	3,684 ms	%10,7	5,508 ms

Tablo 14: S11, 587 pencere/koşu, 4 tur dönüşümlü. Medyanlar arasındaki fark (%15,8) yayılımla aynı mertebede, yani hız farkı ayırt edilemiyor. **Varsayılanın 1 olmasının gerekçesi hız değil kararlılık.**

Yeniden örnekleme nerede yapılmalı — ağ ile hesap arasındaki takas

`native` yolda indirgeme sunucuda yapılır ve **1,57 ms** tutar (toplamın %28,6'sı). Buna karşılık ağdan 500 ms'lik bir tick'te **11 536 bayt** yerine 1 792 bayt geçirmek mümkün olurdu, yani indirgemeyi sunucuda yapmak ağdan **6,4 kat** fazla veri geçirmek demek. Ölçülen toplam bant genişliği yine de yalnızca **187 kbps** — yani bu takasta ağ tarafı ucuz, hesap tarafı pahalı. `native` gerçekçi olan yoldur ve seçilen odur.

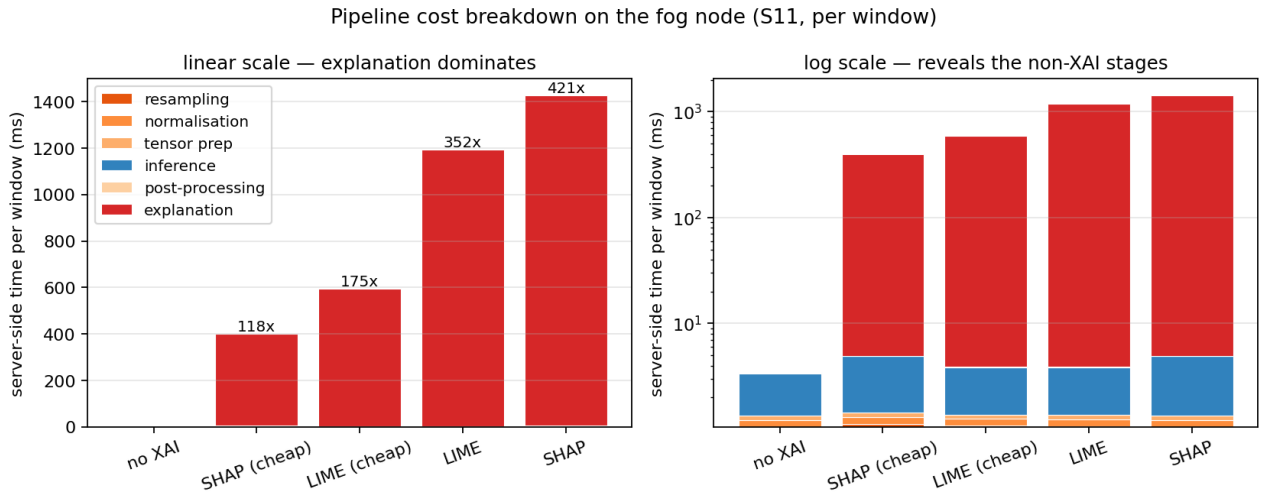
4.2 Açıklama, sistemin baskın maliyeti

Açıklayıcı	xai	execution	response p50	p95	latency p50	CPU	RSS tepe
yok	—	3,59	**10,62**	42,23	4,46	%0,0	210,4
`shap_cheap`	395,03	400,18	**442,61**	574,01	4,32	%5,0	422,3
`lime_cheap`	590,09	594,18	**645,22**	802,47	5,27	%10,8	442,4
`lime`	1188,79	1192,88	**1227,54**	1407,12	4,95	%22,8	449,2
`shap`	1422,28	1427,43	**1456,17**	1548,50	4,52	%25,9	464,5

Tablo 15: Ana tablo. Sis düğümü, S11, tek istemci, 600 tick, 44 ölçülen pencere (ms; CPU tek çekirdeğe göre; RSS MB). "xai" ve "execution" ortalama; response medyan ve p95. Kaynak: `04_fog/fog_xai_*.json`

Dört sonuç

1. Pencere başına toplam sunucu hesabı, açıklayıcı kapalıyken 3,39 ms. Açıkken: `shap_cheap` 400 ms (**118**×), `lime_cheap` 594 ms (**175**×), `lime` 1193 ms (**352**×), `shap` 1427 ms (**421**×). Yanıt süresi cinsinden $10{,}6 \rightarrow 1456$ ms, yani **137 kat**.
2. Ucuz çalışma noktaları 1 s/pencere sınırının altında kalıyor (SHAP 443 ms, LIME 645 ms medyan); pahalı olanlar p95'te aşıyor.
3. Bellek $210 \rightarrow 464$ MB. 2 GB'lık bir Raspberry Pi'de sıkışır; en az 4 GB gerekir.
4. Latency hepsinde 4,3–5,3 ms bandında — açıklayıcı seçimi ağı etkilemiyor. Beklenen davranış ve ölçümün *tutarlılık kontrolü*: latency de değişseydi ölçümde hata olduğunu anlardık.



Şekil 1: Sol panel doğrusal ölçekte: açıklama (kırmızı) bütün grafiği kaplıyor, diğer aşamalar görünmüyor bile. Sağ panel logaritmik ölçekte: aynı veri, ama yeniden örnekleme (turuncu) ve çıkarım (mavi) artık seçilebiliyor. Çubukların üstündeki oranlar açıklayıcısız tabana (3,39 ms) görelidir.

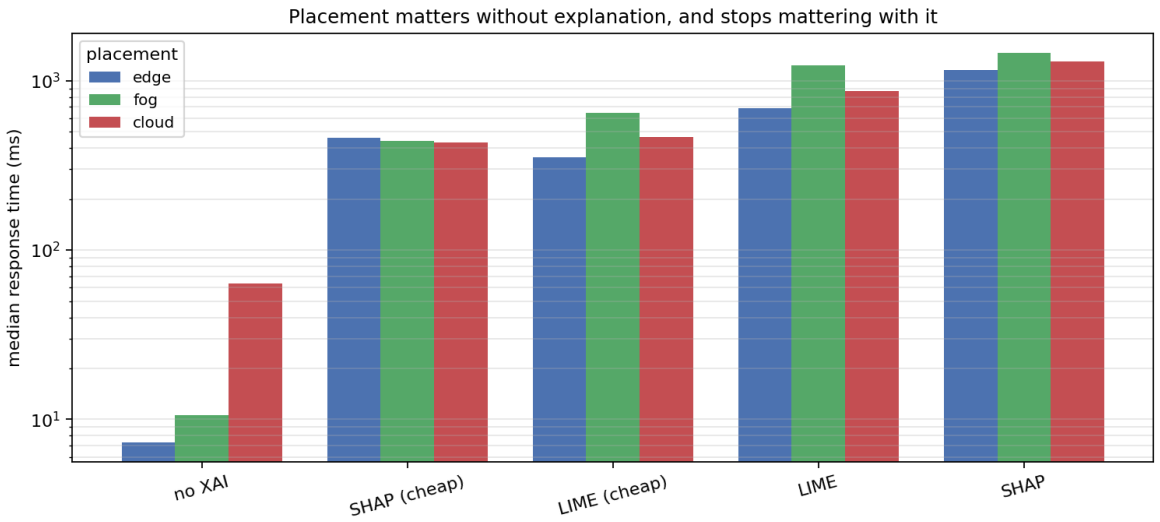
4.3 Yerleşim kararı, açıklama yükü altında anlamını yitiriyor

Açıklayıcı	uç (ortak)	sis (LAN)	bulut (internet)	bulut/uç
yok	7,31	10,62	**63,52**	**8,69**×
`shap_cheap`	462,01	442,61	429,84	0,93×
`lime_cheap`	352,81	645,22	463,26	1,31×
`lime`	690,05	1227,54	870,09	1,26×
`shap`	1154,45	1456,17	1300,46	**1,13**×

Tablo 16: Üç yerleşim yan yana (medyan yanıt süresi, ms).

Raporun en çarpıcı sonucu

Fark ağdan geliyor, hesaptan değil. Ağ gecikmesi uçta ≈1 ms, siste 4–5 ms, bulutta 40–60 ms. Hesap tarafında bulut VPS'i sis düğümünden yavaş *değil* — `shap_cheap`'te hatta biraz hızlı (337'ye karşı 395 ms) ve bunu **tek çekirdekle** yapıyor.
Sonuç: XAI'siz durumda bulut, uçtan 8,7 kat yavaş; `shap` ile yalnızca 1,13 kat — çünkü 50 ms'lik ağ farkı 1200 ms'lik açıklamanın yanında kayboluyor. Bu, referans makalenin *"fog, cloud'dan üstündür"* sonucunu **XAI yükü altında geçersiz kılıyor**.



Şekil 2: Dikey eksen logaritmik. En soldaki grupta üç çubuk arasında büyük fark var (7 / 11 / 64 ms); sağa doğru, yani açıklama pahalılaştıkça çubuklar birbirine yaklaşıyor ve en sağda neredeyse eşitleniyor.

4.4 Eşik nasıl seçildi

Aday eşikler `python -m tools.calibration --tag d1` ile tarandı. Ölçüt: *düşük `explain_rate` ile yüksek `error_recall`*.

eşik	açıklanan	hata yakalama	tetik isabeti	marjinal maliyet
0,60	%6,3 (38 pencere)	0,379 (11/29)	0,289	—
0,70	%10,4 (63)	0,621 (18/29)	0,286	+25 pencere → +7 hata = 3,6
0,80	**%14,4 (87)**	**0,897 (26/29)**	**0,299**	+24 pencere → +8 hata = **3,0**
0,90	%21,4 (129)	1,000 (29/29)	0,225	+42 pencere → +3 hata = **14,0**

Tablo 17: S11 (iyi kalibre, 604 pencere, 29 hata).

eşik	açıklanan	hata yakalama	tetik isabeti
0,60	%1,7 (10)	0,027 (5/184)	0,500
0,70	%4,4 (26)	0,060 (11/184)	0,423
0,80	**%11,8 (70)**	**0,120 (22/184)**	0,314
0,90	%29,3 (174)	0,288 (53/184)	0,305

Tablo 18: S14 (bozuk kalibre, 594 pencere, 184 hata). Aynı eşikte maliyet benzer, *fayda* yedi kat düşük.

Neden 0,80 — üç gerekçe

1. Eğrinin dirseği orada. 0,60'tan 0,80'e kadar her ek hata ≈3 pencereye mal oluyor; 0,80'den 0,90'a geçişte bu **14 pencereye** fırlıyor. Son 3 hatayı yakalamak için 42 fazladan açıklama gerekiyor.
2. Tetik isabeti 0,80'de tepe yapıyor (0,299) ve 0,90'da 0,225'e çöküyor — 0,90 israf bölgesi.
3. %86 tasarruf hâlâ duruyor ve hataların %90'ı yakalanıyor.
Seçim **elle** yapıldı; otomatik bir ölçüt kullanılmadı.

Dört doğrulama koşusu. Eşik seçildikten sonra sunucuda gerçekten koşuldu:

#	Dosya	gerçek eşik	süre	pencere	açıklanan
1	`kisa_segment_sel60.json`	**0,70**	5 dk	44	**0 (%0)**
2	`kisa_segment_sel80.json`	0,80	5 dk	44	**0 (%0)**
3	`kisa_segment_sel90.json`	0,90	5 dk	44	**0 (%0)**
4	**`fog_sel80_tam.json`**	**0,80**	**87 dk**	**1030**	**145 (%14,1)**

İlk üç koşu neden %0 çıktı — ve dosya adları neden yanıltıcı

Hata değil, **kayıt içi durağansızlığın kanıtı**. S11'in ilk 5 dakikasında en düşük güven **0,929**; en yüksek eşik (0,90) bile hiçbir pencerenin altına inmiyor. Düşük güvenli ilk pencere **#100**, yani kaydın 8. dakikasında; kayıttaki 87 düşük güvenli pencerenin hepsi ilk 400 pencere içinde. Seçici XAI deneyi bu yüzden **tam kayıt** üzerinde koşuldu. Bu, "5 dakikalık örnek yeterlidir" varsayımının neden tehlikeli olduğunun somut kanıtıdır.
Ayrıca: `sel60` gerçekte eşik **0,70** ile koşuldu (etiket kayması). Gerçek eşikler sunucu künyesinde (`server.xai.threshold`); **dosya adına değil künyeye bakılmalı**. Eşik 0,60 hiç koşulmadı.

4.5 Seçici açıklama: ana sonuç

Durum	pencere	response ort.	p50	p95	p99
hiç açıklama	44	15,93	10,62	42,23	91,31
seçici (0,80)	**1030**	**80,13**	**10,95**	**445,70**	517,03
hep açıkla	44	450,54	442,61	574,01	599,39

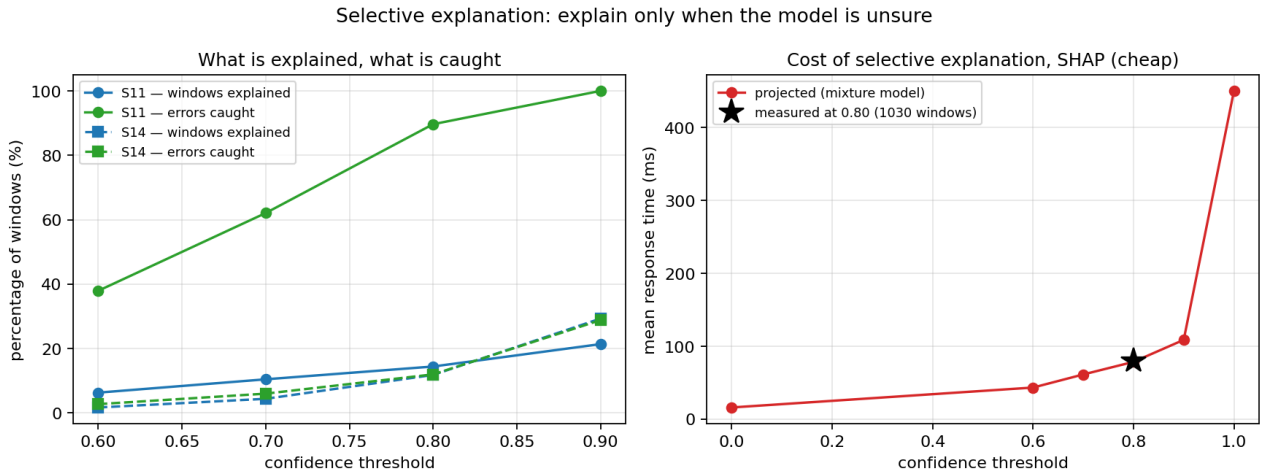
Tablo 19: S11 tam kaydı (10 466 tick, 87 dk), sis düğümü, `shap_cheap` (ms). 1030 pencere = diğer sis koşularının 23 katı, yani p95/p99 burada gerçekten anlamlı. Saat farkı 88 kez yeniden ölçüldü.

Ölçülen sonuç

Seçici açıklama, her pencereyi açıklamaya göre **5,6 kat ucuz** ($450{,}54/80{,}13$). Açıklama oranı %14,1 (145/1030); açıklama başına 364,82 ms, pencere başına amorti edilmiş 51,36 ms. **Medyan yanıt süresi 10,95 ms** — tipik pencere, açıklama hiç yokmuş gibi hızlı cevaplanıyor. Hataların %89,7'si hâlâ açıklanıyor.
Kuyrukta birikme YOK: queuing ortalama 0,031 ms, p99 0,057 ms. Patlamalı yükün (%86 pencere 3,6 ms, %14 pencere ≈365 ms) kuyruğu bozması beklenebilirdi; bozmadı. Sebebi: hop bütçesi 5000 ms ve en pahalı pencere bile 365 ms sürüyor, yani bir sonraki pencere gelmeden iş bitiyor.

	projeksiyon	ölçüm	hata
açıklama oranı	%14,4	%14,1	**%2,3**
ortalama response	78,53 ms	80,13 ms	**%2,0**
p95 response	487,25 ms	445,70 ms	%8,5

Tablo 20: Karışım modeli ölçümle doğrulandı. Ortalamadaki %2 beklenen (karışımın ortalaması matematiksel olarak kesindir); p95'teki %8,5, özet çeyrekliklerden ara değerle tahmin edildiği için baştan "yaklaşık" diye işaretliydi. **Sonuç:** dosyadaki diğer 44 satır (öteki açıklayıcılar, S14) güvenle projeksiyon olarak sunulabilir.



Şekil 3: Sol panel: eşik yükseldikçe açıklanan pencere oranı (mavi) yavaş, yakalanan hata oranı (yeşil) hızlı artıyor. S14'te (kesikli) aynı eğri çok daha düşük — kalibrasyon bozuk olduğu için hata yakalama %12'de kalıyor. Sağ panel: kırmızı eğri karışım modelinin projeksiyonu, siyah yıldız **1030 pencerelik gerçek ölçüm**; ikisi üst üste düşüyor.

4.6 Yük altında kapasite

Durum	istemci	queuing ort.	q p95	execution	response ort.	p95
XAI yok	1	0,030	0,035	3,59	15,93	42,23
XAI yok	2	1,607	4,514	3,57	24,31	107,78
XAI yok	4	4,555	12,505	3,69	28,50	120,27
hep açıkla	1	0,032	0,052	400,18	450,54	574,01
hep açıkla	2	**160,12**	381,58	371,06	601,30	853,58
hep açıkla	4	**437,25**	**1125,11**	370,75	984,94	1643,07

Tablo 21: Sis düğümü, `shap_cheap`, S11+S14 döngüsel, 600 tick (ms). Bulut karşılığı (4 istemci, XAI yok, 4120 pencere): queuing ort. 4,74, p95 16,08; response ort. 67,48, p95 117,81 ms.

Durum	execution	kapasite (5000/exec)
XAI yok	3,59 ms	**1394** istemci
seçici (0,80)	55,26 ms	**90** istemci
hep açıkla	400,18 ms	**12** istemci
referans makale, tek worker		≈20 istemci

Tablo 22: Seçici açıklama kapasiteyi **7,5 kat** artırıyor.

4.7 Açıklayıcı seçimi: “daha iyi” yok, denge var

Nokta	gecikme	faithfulness ± SEM	monotonicity	sensitivity ± SEM
`shap_cheap`	395 ms	0,2499 ± 0,0426	0,5650	0,1228 ± 0,0032
`lime_cheap`	590 ms	0,2092 ± 0,0345	0,6038	**0,0489** ± 0,0016
`lime`	1189 ms	0,2324 ± 0,0373	0,5788	**0,0446** ± 0,0018
`shap`	1422 ms	0,2632 ± 0,0445	0,5488	0,1282 ± 0,0046

Tablo 23: S11, 40 pencere. Kaynak: `11_pareto/pareto_c5.json`

Üç sonuç

1. Faithfulness’ta hiçbir nokta diğerini yenmiyor — dört nokta 0,209–0,263 aralığında, SEM ≈0,04, farkların hepsi hata payının içinde. *Seçim faithfulness’a göre yapılamaz.*
2. Ayrışma sensitivity’de ve kesin: LIME 2,7× daha kararlı, fark hata payının 15 katı.
3. Gerçek denge: SHAP 3× ucuz, LIME 2,7× kararlı, sadakat eşit.
Örnek başına maliyet tersine dönüyor: LIME 2,4 ms/örnek $(590/250)$, SHAP 7,9 ms/örnek $(395/50)$. Tek örnek düzeyinde SHAP pahalı — literatürle uyumlu — ama çok daha az örnekle yetindiği için toplamda ucuz. Bu ayrım makalede yazılmalı, yoksa çelişkili görünür.

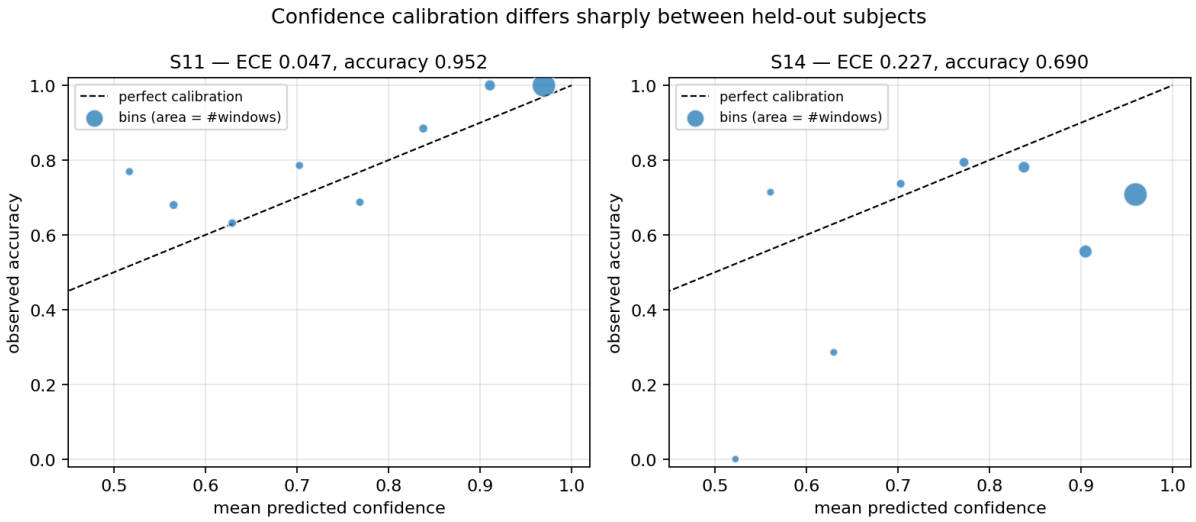
4.8 Faydanın koşulu: kalibrasyon

	S11	S14
pencere / hata sayısı	604 / 29	594 / 184
doğruluk	**0,9520**	**0,6902**
ortalama güven	0,9093	0,9085
fazla özgüven	−0,0426	**+0,2182**
ECE	**0,0469**	**0,2265**
eşik 0,80’de açıklanan	%14,4	%11,8
eşik 0,80’de hata yakalama	**0,897**	**0,120**

Tablo 24: İki tutulmuş denek taban tabana zıt.

İki iddia ayrı yazılmalı

Model **ikisine de aynı güvenle yaklaşıyor** (0,909) ama birinde %95, diğerinde %69 haklı. S14’te *kendinden emin biçimde yanılıyor*.
Maliyet tasarrufu her iki denekte gerçekleşiyor (açıklama oranı %12–14, kalibrasyondan bağımsız). **Fayda ise kalibrasyona bağlı** (hata yakalama %90’a karşı %12).
Sıcaklık ölçekleme çalışmadı: çapraz denek uygulaması her iki yönde de kötüleştirdi (S11 ECE $0{,}047 \to 0{,}256$; S14 $0{,}227 \to 0{,}276$). Bozukluk zıt yönlerde olduğu için tek parametre ikisini birden düzeltemiyor. Bu bir hata değil, raporlanacak bir bulgudur.



Şekil 4: Kesikli köşegen mükemmel kalibrasyon; baloncuk alanı o kutudaki pencere sayısı. S11 (sol) noktaları köşegenin üstünde veya yakınında. S14 (sağ) en büyük baloncuk köşegenin *altında*: model %96 güvenle konuşuyor ama %71 haklı.

4.9 Referans makale ile karşılaştırma

Metrik	Makale (worker)	Bu çalışma	Karşılaştırılabilir mi?
latency	4,7 ms	0,81 ms(loopback)	hayır — farklı ağ
queuing delay	<1 ms	0,045 ms	hayır
execution time	34,7 ms	7,18 ms	hayır — Pi 4 vs masaüstü
response time	43,3 ms	8,82 ms	hayır
jitter	17,4 ms	3,53 ms	hayır
bandwidth	**374 kbps**	**187 kbps**	**evet**

Tablo 25: Erzurumluoğlu ve ark., ICACT 2025 (DREAMER, Raspberry Pi 4 worker).

Anlamlı olan tek karşılaştırma bant genişliği

Onlar 3 saniyelik pencereden 24 öznitelik çıkarıp gönderiyor, biz 10 saniyelik pencerenin **ham sinyalini** gönderiyoruz — buna rağmen **yarısı kadar** veri geçiyor. Asıl fark mimari: onlarda öznitelik çıkarımı hesabın çoğunu oluşturuyor, bizde ham sinyal doğrudan modele giriyor.
Pi 4 çıkarımı: referansın Pi 4'ünde execution 34,7 ms, bizde 3,6 ms — ≈10 kat. Aynı oran açıklamaya uygulanırsa `shap_cheap` bir Pi 4'te ≈**4 saniye** eder ve 5 saniyelik hop bütçesini neredeyse doldurur. Yani gerçek sis donanımında seçici açıklama *zorunlu* hâle gelir.

5 Uyarılar ve sınırlılıklar

5.1 Makaleye sayı taşırken dikkat

1. **Sis XAI koşuları 44 pencere, tek tekrar.** Ortalama karşılaştırması geçerli; **p95/p99 yalnızca 1030 pencerelik seçici koşudan** alınmalı.
2. **Sis koşularındaki sınıflandırma sayıları anlamsız.** S11'in ilk 5 dakikasında hiç stres etiketi yok (42 pencerenin hepsi stres-olmayan, doğruluk 1,00). Başarım Çizelge 5'ten (Bölüm 4.1) okunmalı.

3. **Yazılım yığını yerleşimler arasında farklı.** Uç: Python 3.11 / PyTorch 2.8. Sis: Python 3.14 / PyTorch 2.14. Bulut: Docker python:3.11 / PyTorch 2.14. Bu yüzden `07_edge_xai` ile `04_fog` arasındaki sıralama tutarsız: LIME'da uç hızlı (390'a karşı 590 ms), SHAP'ın ucuz noktasında sis hızlı (395'e karşı 417 ms), tabanda sis hızlı. **Donanım farkı böyle davranmaz** — olduğu gibi "sis daha yavaş" diye sunulamaz. Kapatmak için sis düğümünü Docker'la (python:3.11) yeniden koşmak yeterli (≈25 dk).

4. **Eşik, üzerinde test edildiği denekte seçildi.** τ=0,80, S11'in tablosuna bakılarak seçildi ve ana doğrulama koşusu da S11'de yapıldı. `PLAN.md` kural 8 ("eşik seçimi test deneğinde yapılmaz") bu koşuda uygulanmadı. S14 için eşik 0,80 yalnızca *projeksiyon* olarak var. Kapatmak için S14 tam kayıt koşusu (≈5 dk) ya da sınırlılıklarda açık ifade.

5. **`results/README.md` ile ham dosyalar arasında beş fark bulundu.** En önemlisi: `benchmark_xai_cpu.json`'da "LIME 250 örnek p50 490 ms" diye bir kayıt **yok**; dosyadaki iki kayıt 500 örnek/20 dilim (p50 710 ms) ve 500 örnek/**5** dilim (p50 678 ms). Ayrıca `05_yerlesim` sayıları farklı (uç 0,928 / 6,344 / 7,880; sis 4,910 / 3,372 / 10,151) ve `01_streaming` "1175 pencere" değil 2123 ölçülen / 1198 etiketli. Tam liste ayrıntılı raporun 12. bölümünde. **Makaleye sayı doğrudan JSON'dan okunmalı.**

6. **`README.md`'nin "Ölçülen sonuçlar" tablosu eski** (preprocessing 0,189 ms, inference 1,15 ms, toplam 1,27 ms) — `prepared` yol dönemine ait.

5.2 Sınırlılıklar

1. **İki tutulmuş denek (n=2).** Kalibrasyon bulgusu ve "denek bağımlı" sonucu genelleme değil, gözlemdir.

2. **Sis düğümü bir masaüstü, Raspberry Pi değil.** Ölçülen şey *dağıtık yerleşim*, *kaynak kısıtlı sis düğümü* değil. Tek çekirdeğe kilitlemenin bu iş yükünde hiçbir şey değiştirmediği ölçüldü (süreç zaten tek iplik, tepe CPU %105–111); bağlayıcı tek yöntem kesirli CPU kotasıdır (Docker `--cpus=0.1`).

3. **Kuyruk ölçümü bir üst sınırdır** (istemciler faz hizalı başlıyor).

4. **Latency Wi-Fi ve internette belirsiz** ($\pm$1–1,7 ms / $\pm$25–51 ms). Execution, queuing, response ve jitter bundan etkilenmez.

5. **Emülasyon, canlı sistem değil.** WESAD kaydından replay yapılıyor — referans makalenin de yaptığı budur, ama "clinical" veya "live system" denmemeli.

6. **Tek model, iki açıklayıcı**; kullanıcı çalışması yok; MCU analizi kâğıt üstünde.

6 Kaynak haritası ve komutlar

Sonuç	Kaynak
Aşama bazlı hesaplama profili	`01_streaming/streaming_summary_native_fixed.json`
İplik / ardışık düzen taraması	`02_benchmark/benchmark_thread_sweep.json`, `benchmark_pipeline_cmp.json`
Açıklayıcıların istemci CPU maliyeti	`02_benchmark/benchmark_xai_cpu.json`
Yedi metrik, loopback	`03_loopback/fog_realtime_1user.json`
Ana maliyet tablosu	`04_fog/fog_xai_*.json`
Uç vs sis (XAI'siz)	`05_yerlesim/deployments_edge_vs_fog.json`
XAI hiperparametre taraması	`06_xai_tarama/*.csv`
Ortak yerleşimde XAI	`07_edge_xai/edge_xai_*.json`
Kalibrasyon, ECE, eşik tablosu	`08_kalibrasyon/calibration_d1.json`
Seçici açıklama, ana koşu	`09_secici_xai/fog_sel80_tam.json`
Eşik projeksiyonu (48 satır)	`09_secici_xai/selective_xai_sweep_d2.json`
Çok istemcili yük ve kapasite	`10_yuk/fog_load_*.json`
Kalite–gecikme Pareto	`11_pareto/pareto_c5.json`
Aşama profili derlemesi	`12_profil/profile_table_b1.json`
Bulut yerleşimi	`13_bulut/cloud_*.json`
Şekiller	`14_figures/` (`python -m tools.figures`)
Seçilmiş XAI yapılandırmaları	`checkpoints/xai_config.json`
SHAP arka plan kümesi	`checkpoints/shap_background_k100_kmeans.npz`
Geçersiz sayılan ölçümler	`results/eski_ve_hatali/` (+ kendi README'si)

```
# Sunucuyu ikinci makinede başlat (seçici XAI ile)
python -m serving.server --host 0.0.0.0 --port 9099 \
    --xai-method shap_cheap --xai-threshold 0.8

# İstemci (verinin olduğu makinede)
python run_fog_experiment.py --server-host 192.168.1.14 \
    --subject S11 --realtime --tag sel80_tam

# Yerleşim karşılaştırması (dönüşümlü, 3 tekrar)
python compare_deployments.py --subject S11 --realtime --max-ticks 600 \
    --repeat 3 --deployment edge=local fog=192.168.1.14 --tag edge_vs_fog

# Tekrarlı, dönüşümlü profil ölçümü
python benchmark_runs.py --subject S11 --repeat 4 --threads 1 2 4 --tag thread_sweep

# Türetilmiş çıktılar
python -m tools.calibration --tag d1
python -m tools.selective_sweep --tag d2
python -m tools.pareto --tag c5
python -m tools.profile_table
python -m tools.figures
python -m tools.mcu_feasibility

# Docker ile bulut dağıtımı (sunucu WESAD verisine ihtiyaç duymaz)
WESAD_XAI_METHOD=shap_cheap WESAD_XAI_THRESHOLD=0.8 docker compose up -d --build
```

Ayrıntılı sürüm: `docs/teknik_rapor.pdf` — tam sözlük, 13 deneyin tek tek dökümü, geçersiz sayılan ölçümlerin gerekçeleri, bitirme projesi sonuçları ve tespit edilen tutarsızlıkların tam listesi.